Desarrollo de Aplicación Híbridas

UT3: Diseño de Interfaces de Usuario

UT3: Diseño de Interfaces de Usuario

¡Bienvenido/a a la Unidad de Trabajo 3! Una vez que hemos sentado las bases de cómo funcionan los componentes y cómo mostrar datos con Angular, es el momento de centrarnos en un aspecto fundamental de cualquier aplicación: su **apariencia y usabilidad**. Una aplicación no solo tiene que funcionar bien, también tiene que ser atractiva, intuitiva y fácil de usar en cualquier dispositivo.

En esta unidad, nos sumergiremos de lleno en las herramientas que Ionic nos proporciona para construir interfaces de usuario profesionales. Aprenderemos a estructurar el contenido en la pantalla, a personalizar los estilos para que se ajusten a la identidad de nuestro proyecto y a utilizar componentes interactivos esenciales para comunicarnos con el usuario.

Objetivos de Aprendizaje

Al finalizar esta unidad, serás capaz de:

- Maquetar la estructura de tus vistas usando el **sistema de rejilla (Grid)** de Ionic.
- Crear **diseños adaptativos (responsive)** que se vean bien tanto en pantallas de móvil como de escritorio.
- Utilizar las **utilidades de Flexbox** para alinear y distribuir elementos con precisión.
- Personalizar los colores y la apariencia general de tu aplicación mediante el sistema de **Theming con variables CSS**.
- Implementar y dar estilo al **modo oscuro (Dark Mode)**.
- Utilizar los **componentes de formulario** más comunes para recoger datos del usuario (inputs, selects, toggles).
- Mostrar **notificaciones y ventanas emergentes** (Alerts, Toasts y Modals) para interactuar con el usuario.

Esta unidad es clave para que tu proyecto personal empiece a tomar forma y pase de ser un esqueleto funcional a una aplicación con una interfaz pulida y profesional.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.1. El Sistema de Rejilla (Grid) de Ionic

3.1. El Sistema de Rejilla (Grid) de Ionic

Cuando empezamos a diseñar una pantalla, nuestra primera tarea es decidir dónde irá cada elemento. ¿Este texto va a la izquierda? ¿Esta imagen ocupa toda la parte superior? Para organizar el espacio de forma ordenada y predecible, Ionic nos proporciona un **sistema de rejilla (Grid)** muy potente y fácil de usar.

¿Cómo funciona el Grid? La analogía de la hoja de cálculo

Imagina una hoja de cálculo. Toda la hoja es la **rejilla (`ion-grid`)**. Dentro, tienes **filas (`ion-row`)** que van de arriba a abajo. Y cada fila se divide en 12 **columnas (`ion-col`)** invisibles. Tú decides cuántas de esas 12 columnas ocupa cada elemento.

Los tres componentes que trabajan juntos son:

- **`<ion-grid>`**: Es el contenedor principal que envuelve toda la estructura.
- **`<ion-row>`**: Define una fila horizontal. Todos los elementos que quieras que estén en la misma línea deben ir dentro de una fila.
- **`<ion-col>`**: Define una columna. Aquí es donde colocas tu contenido (texto, imágenes, botones...). Por defecto, si pones dos columnas en una fila, cada una ocupará la mitad del espacio (6 de 12 columnas).

Ejemplo Práctico: Una Tarjeta de Perfil Simple

Para construir nuestro ejemplo, necesitamos actualizar tanto la vista (HTML) como la lógica (TypeScript) de nuestra página `home`.

1. La Lógica en `home.page.ts`

Partiendo del archivo que genera Ionic por defecto, debemos añadir los componentes del sistema de Grid (`IonGrid`, `IonRow`, `IonCol`) y el botón (`IonButton`) para que la plantilla HTML pueda reconocerlos.

```
// Archivo: src/app/home/home.page.ts

import { Component } from '@angular/core';
import { IonHeader, IonToolbar, IonTitle, IonContent } from '@ionic/angular/st
// 1. Añadimos aquí los nuevos componentes que vamos a usar
import { IonGrid, IonRow, IonCol, IonButton } from '@ionic/angular/standalone'

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  // 2. Y los registramos también en el array de imports del componente
  imports: [IonHeader, IonToolbar, IonTitle, IonContent,
    IonGrid, IonRow, IonCol, IonButton],
})
export class HomePage {
  constructor() {}
}
```

2. La Vista en `home.page.html`

Ahora, modificamos el archivo HTML para integrar nuestra maquetación con el sistema de rejilla.

```
<!-- Archivo: src/app/home/home.page.html -->

<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>
      Perfil de Usuario
    </ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true" class="ion-padding">
  <ion-header collapse="condense">
    <ion-toolbar>
      <ion-title size="large">Perfil de Usuario</ion-title>
    </ion-toolbar>
```

```
    </ion-header>

    <!-- Aquí empieza nuestro contenido personalizado -->
    <ion-grid>
      <!-- Fila 1: Imagen a la izquierda, texto a la derecha -->
      <ion-row>
        <!-- Columna 1: Ocupará la mitad del espacio disponible -->
        <ion-col>
          <img src="https://placehold.co/400x400/8d6e63/ffffff?text=Avatar" alt=

        </ion-col>

        <!-- Columna 2: Ocupará la otra mitad del espacio -->
        <ion-col>
          <p>Este es el texto descriptivo del perfil. Al estar en una columna se
          <ion-button>Contactar</ion-button>
        </ion-col>
      </ion-row>
    </ion-grid>

  </ion-content>
```

Controlando el Ancho de las Columnas con `size`

Por defecto, las columnas dentro de una fila se reparten el espacio a partes iguales. Pero, ¿y si queremos que la imagen ocupe menos espacio que el texto? Para eso, usamos la propiedad **`size`**.

Recuerda que cada fila tiene 12 unidades imaginarias. Podemos decirle a una columna cuántas de esas 12 unidades debe ocupar.

Modificamos nuestra `<ion-row>` en `home.page.html`:

```
  <!-- ... dentro de la ion-grid ... -->
  <ion-row>
    <!-- Esta columna ocupará 4 de las 12 unidades (un tercio) -->
    <ion-col size="4">
      <img src="https://placehold.co/400x400/8d6e63/ffffff?text=Avatar" alt="Ava
    </ion-col>

    <!-- Esta columna ocupará 8 de las 12 unidades (dos tercios) -->
    <ion-col size="8">
      <p>Este es el texto descriptivo del perfil. Ahora tiene más espacio dispor
```

```
        <ion-button>Contactar</ion-button>
      </ion-col>
    </ion-row>
```

¡Experimenta con diferentes valores en la propiedad `size` (siempre que sumen 12 o menos por fila) para ver cómo afecta a la maquetación!

> ### ✅ Práctica Rápida
>
> Intenta replicar esta estructura de tarjeta de perfil en la página `home` de tu proyecto personal, pero adaptándola a tu temática. Por ejemplo, si tu app es de recetas, podrías poner la foto de la receta a la izquierda y los ingredientes a la derecha.
>
> **¿Cómo pongo una imagen?** Para esta práctica, la forma más sencilla es usar una **URL de una imagen que encuentres en internet**. Busca una foto, haz clic derecho sobre ella y selecciona "Copiar dirección de la imagen". Luego, pega esa URL en el atributo `src` de tu etiqueta `<img>`. Más adelante veremos cómo usar imágenes guardadas localmente en nuestro proyecto.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.2. Diseño Adaptativo con la Grid Responsiva

3.2. Diseño Adaptativo con la Grid Responsiva

En la sección anterior, aprendimos a maquetar nuestra vista usando filas y columnas. Pero, ¿qué pasa cuando vemos esa misma maquetación en una pantalla más grande, como la de una tablet o un ordenador? Una estructura que se ve perfecta en un móvil puede parecer vacía y desaprovechada en una pantalla ancha.

Aquí es donde entra en juego el **diseño adaptativo (o responsivo)**. El objetivo es que nuestra interfaz se adapte automáticamente al tamaño de la pantalla, mostrando una maquetación optimizada para cada dispositivo.

Puntos de Ruptura (Breakpoints)

Para lograrlo, Ionic nos proporciona unos "puntos de ruptura" (breakpoints). Son tamaños de pantalla predefinidos que podemos usar como referencia para aplicar diferentes estilos. Los principales son:

- `xs` **(extra small):** Para teléfonos móviles (por defecto).
- `sm` **(small):** Para teléfonos grandes o tablets en vertical.
- `md` **(medium):** Para tablets en horizontal.
- `lg` **(large):** Para portátiles y monitores de escritorio pequeños.
- `xl` **(extra large):** Para monitores grandes.

Aplicando Breakpoints a las Columnas

La magia ocurre cuando combinamos estos breakpoints con la propiedad `size` que ya conocemos. En lugar de usar `size="..."`, que se aplica a todos los tamaños, podemos usar `size-md="..."`, que solo se aplicará en pantallas de tamaño "medium" en adelante.

La regla es: Ionic aplica el `size` del breakpoint más pequeño que corresponda. Si no hay ninguno, aplica el `size` por defecto.

Ejemplo Práctico: Haciendo nuestro Perfil Responsivo

Vamos a mejorar el perfil de usuario que creamos en la sección anterior. Queremos que:

1. En pantallas pequeñas (móvil), la imagen y el texto se apilen uno encima del otro.
2. En pantallas medianas o grandes (tablet/escritorio), se muestren uno al lado del otro.

Para lograr esto, no necesitamos modificar el archivo `home.page.ts`, ¡solo el HTML!

Modificamos nuestra `<ion-row>` en `home.page.html`:

```
<!-- Archivo: src/app/home/home.page.html -->
<!-- ... dentro de la ion-grid ... -->
<ion-row>
  <!--
    Por defecto (xs), esta columna ocupa las 12 unidades (ancho completo).
    A partir del tamaño 'md', ocupará solo 4 unidades.
  -->
  <ion-col size="12" size-md="4">
    <img src="https://placehold.co/400x400/8d6e63/ffffff?text=Avatar" alt="Ava
  </ion-col>

  <!--
    Por defecto (xs), esta columna también ocupa las 12 unidades, apilándose d
    A partir del tamaño 'md', ocupará las 8 unidades restantes.
  -->
  <ion-col size="12" size-md="8">
    <p>Este es el texto descriptivo del perfil. Ahora tiene más espacio dispor
    <ion-button>Contactar</ion-button>
  </ion-col>
</ion-row>
```

¿Cómo Pruebo el Diseño Adaptativo?

Ahora que has añadido las clases responsivas, necesitas una forma de ver cómo se comporta tu aplicación en diferentes tamaños de pantalla. La forma más sencilla es **cambiar el tamaño de la ventana de tu navegador**. Hazla más estrecha y más ancha y

> verás en tiempo real cómo la maquetación "salta" de la vista de una columna a la de dos al pasar por el punto de ruptura "md".

Uso Avanzado: Combinando `*ngFor` con la Grid Responsiva

En la Unidad 2, vimos cómo usar `*ngFor` para repetir un componente, como nuestro `<app-task-item>`, creando una lista vertical:

```
<!-- Así lo hacíamos en la UT2 -->
<app-task-item
  *ngFor="let tarea of listaDeTareas"
  [tarea]="tarea">
</app-task-item>
```

Pero, ¿qué pasa si queremos que esa lista se muestre en formato de rejilla, con varios elementos por fila en pantallas grandes? Aquí es donde combinamos el poder de `*ngFor` con `<ion-col>`.

La clave es colocar la directiva `*ngFor` **directamente en la etiqueta `<ion-col>`**. De esta manera, Angular creará una nueva columna por cada elemento de la lista, y nosotros podemos usar las propiedades `size` responsivas para controlar cuántas columnas caben en cada fila según el tamaño de la pantalla.

Ejemplo práctico para mostrar nuestra lista de tareas en una rejilla responsiva:

```
<!-- En home.page.html, reemplazando el *ngFor anterior -->
<ion-grid>
  <ion-row>
    <!--
      - Por cada 'tarea' en 'listaDeTareas', crea una columna (<ion-col>).
      - En móvil (xs), cada columna ocupa 6/12 (mitad de pantalla -> 2 columna
      - En tablets (md), cada columna ocupa 4/12 (un tercio -> 3 columnas por
      - En escritorio (lg), cada columna ocupa 3/12 (un cuarto -> 4 columnas p
    -->
    <ion-col *ngFor="let tarea of listaDeTareas" size="6" size-md="4" size-lg=
      <!-- Dentro de cada columna, renderizamos nuestro componente de tarea --
      <app-task-item [tarea]="tarea"></app-task-item>
    </ion-col>
```

```
      </ion-row>
    </ion-grid>
```

Con esta estructura, tu lista de tareas se adaptará elegantemente, mostrando más información en pantallas grandes y aprovechando mejor el espacio, sin dejar de ser perfectamente usable en móviles.

> ### ✅ Práctica Rápida
>
> Aplica esta misma técnica a la maquetación que hiciste para tu proyecto personal. Juega con diferentes breakpoints (`size-sm`, `size-lg`) y con diferentes valores de `size` para ver cómo se comporta tu interfaz.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.3. Alineación Fina con Utilidades Flexbox

3.3. Alineación Fina con Utilidades Flexbox

El sistema de rejilla (Grid) es fantástico para definir la estructura principal de nuestra página, pero a veces necesitamos un control más preciso sobre cómo se alinean los elementos *dentro* de una columna. Por ejemplo, ¿cómo centramos verticalmente un botón y un texto? ¿O cómo ponemos un elemento a la derecha del todo?

Para estos casos, Ionic nos ofrece un conjunto de **clases de utilidad basadas en Flexbox**, un modelo de diseño de CSS increíblemente potente para alinear y distribuir el espacio entre ítems.

Grid vs. Flexbox: ¿Cuándo usar cada uno?

Es una pregunta muy común. Piensa en ellos de esta manera:

- Usa el **Grid (`ion-grid`)** para la **maquetación principal** de la página (2D): definir filas y columnas que contienen grandes bloques de contenido. Es como construir las paredes de una habitación.
- Usa las **clases de Flexbox** para **alinear los elementos** que hay dentro de un contenedor, como una columna (`ion-col`) o un `div` (1D). Es como decidir dónde cuelgas los cuadros en una pared.

Las Clases de Utilidad de Flexbox más Comunes

Podemos añadir estas clases a los componentes de la rejilla (como `<ion-row>` o `<ion-col>`) para controlar la alineación:

- **Para alinear horizontalmente (eje principal):**
  - `.ion-justify-content-start`: Alinea los elementos al principio (izquierda). (Por defecto)
  - `.ion-justify-content-center`: Centra los elementos.
  - `.ion-justify-content-end`: Alinea los elementos al final (derecha).

    - `.ion-justify-content-between`: Distribuye los elementos, dejando un espacio máximo entre ellos.
- **Para alinear verticalmente (eje secundario):**
    - `.ion-align-items-start`: Alinea los elementos arriba.
    - `.ion-align-items-center`: Centra los elementos verticalmente.
    - `.ion-align-items-end`: Alinea los elementos abajo.

Otras Clases de Utilidad: Alineación de Texto

Además de las clases de Flexbox para alinear contenedores, Ionic nos da clases sencillas para alinear el texto *dentro* de un elemento:

- `.ion-text-left`: Alinea el texto a la izquierda.
- `.ion-text-center`: Centra el texto.
- `.ion-text-right`: Alinea el texto a la derecha.
- `.ion-text-justify`: Justifica el texto.

> ### Herramienta Clave: El Modo Móvil del Navegador
>
> Ahora que estamos trabajando con clases responsivas, necesitas una forma de comprobar cómo se ve tu aplicación en diferentes tamaños de pantalla sin tener que usar un móvil, una tablet y un ordenador a la vez. ¡Para eso están las **Herramientas de Desarrollador** de tu navegador!
>
> 1. En tu navegador (Chrome, Firefox, Edge...), haz clic derecho en cualquier parte de tu aplicación y selecciona **"Inspeccionar"**. También puedes pulsar la tecla **F12**.
> 2. Se abrirá un panel. En la esquina superior izquierda de ese panel, busca y haz clic en un icono que parece un teléfono y una tablet juntos. Esto activará el **"Modo de Vista de Dispositivo"**.
>
> Una vez activado, tu vista cambiará a un simulador de dispositivo móvil. Ahora puedes:
>
> - **Elegir un dispositivo:** En la parte superior, verás un menú desplegable para simular dispositivos populares como "iPhone 12", "Pixel 5", etc.
> - **Cambiar el tamaño:** Puedes arrastrar los bordes de la vista para cambiar el ancho y ver en tiempo real cómo tu diseño se adapta y cómo saltan los breakpoints que hemos aprendido (`xs`, `sm`, `md`...).
> - **Cambiar la orientación:** Hay un botón para rotar la pantalla y ver cómo se comporta tu diseño en modo horizontal.
>
> Esta herramienta es **esencial** para el desarrollo de aplicaciones híbridas y te permitirá probar todos los ejemplos de diseño adaptativo de forma rápida y sencilla.

Ejemplo Práctico: Mejorando nuestro Perfil

Vamos a mejorar la columna de texto de nuestro perfil. Para que el centrado vertical se aprecie, debemos aplicarlo al contenedor padre, es decir, a la fila (`ion-row`). Al añadir la clase `.ion-align-items-center` a la fila, esta se encarga de centrar verticalmente a todos sus hijos (las columnas).

Modificamos nuestra `<ion-row>` en `home.page.html`:

```
<!-- Archivo: src/app/home/home.page.html -->
<!-- ... dentro de la ion-grid ... -->

<!-- Al añadir .ion-align-items-center a la fila, centramos verticalmente amba
<ion-row class="ion-align-items-center">
  <!-- Columna de la imagen -->
  <ion-col size="12" size-md="4">
    <img src="https://placehold.co/400x400/8d6e63/ffffff?text=Avatar" alt="Ava
  </ion-col>

  <!-- Columna de texto -->
  <ion-col size="12" size-md="8">
    <div class="ion-text-center">
      <p>Este es el texto descriptivo del perfil. Ahora está perfectamente cer
      <ion-button>Contactar</ion-button>
    </div>
  </ion-col>
</ion-row>
```

Con este cambio, la fila se hace tan alta como la imagen y luego centra verticalmente la columna del texto (que es menos alta) para que su contenido aparezca alineado con el centro de la imagen.

> ### ✅ Práctica Rápida
>
> Experimenta con las clases de utilidad de Flexbox en tu proyecto. Prueba a alinear los elementos de una columna al final (abajo a la derecha) usando `.ion-justify-content-end` y `.ion-align-items-end`.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.4. Theming con Variables CSS

3.4. Theming con Variables CSS

Ahora que ya sabemos cómo estructurar nuestra aplicación, vamos a darle una identidad visual única. Ionic tiene un sistema de "theming" (tematización) muy potente basado en **variables CSS**, lo que nos permite cambiar la apariencia de toda la aplicación modificando unas pocas líneas de código.

Los Archivos Clave del Theming

Para personalizar la apariencia de nuestra aplicación, trabajaremos principalmente con dos archivos:

- **`src/theme/variables.scss`**: Este es el centro de control de las **variables**. Aquí es donde definimos nuestra paleta de colores y otras variables globales que controlan la apariencia general.
- **`src/global.scss`**: Este archivo es para escribir **reglas de CSS** que queremos que se apliquen en toda la aplicación, como estilos para etiquetas HTML (`h1`, `p`...) o para sobrescribir los estilos de los componentes de Ionic de forma global.

Conceptos Fundamentales de las Variables CSS

> #### 1. ¿Qué es el selector `:root`?
>
> En CSS, `:root` hace referencia al elemento raíz del documento. Al declarar nuestras variables dentro de este selector en el archivo `variables.scss`, las estamos convirtiendo en **variables globales**, lo que significa que estarán disponibles en cualquier parte de nuestra aplicación.

2. ¿Cómo se "usan" las variables? Con la función `var()`

Una vez que una variable está definida (ej: `--ion-color-primary`), puedes usar su valor en cualquier otra regla CSS utilizando la función **`var()`**. La sintaxis es muy sencilla: `var(--nombre-de-la-variable)`.

Por ejemplo, si en `global.scss` escribimos la siguiente regla, todas las etiquetas `h2` de la app pasarán a tener el color que hayamos definido en la variable `--ion-color-primary`.

```
/* Contenido para añadir en: src/global.scss */
h2 {
  color: var(--ion-color-primary);
}
```

Personalizando la Paleta de Colores

El primer paso para personalizar el tema es definir la paleta de colores principal de la aplicación.

Importante: Un Archivo Inicialmente Vacío

En las versiones más recientes de Ionic, es muy probable que al abrir `variables.scss` lo encuentres vacío. ¡No te preocupes, es normal! Para empezar a personalizar, simplemente tenemos que añadir nuestras propias definiciones de variables.

Anatomía de una Variable de Color

Cada color que definimos en Ionic no es un solo valor, sino una familia de variables relacionadas. Este es un ejemplo completo que deberías añadir a tu `variables.scss` para el color primario:

```scss
/* En src/theme/variables.scss, dentro de :root */
:root {
  --ion-color-primary: #f48c06;
  --ion-color-primary-rgb: 244, 140, 6;
  --ion-color-primary-contrast: #000000;
  --ion-color-primary-contrast-rgb: 0, 0, 0;
  --ion-color-primary-shade: #d77b05;
  --ion-color-primary-tint: #f5981f;
}
```

- **`--ion-color-primary`**: El **color base**.
- **`--ion-color-primary-rgb`**: El color en formato **R, G, B**.
- **`--ion-color-primary-contrast`**: El **color del texto** que irá encima.
- **`--ion-color-primary-shade`**: Una versión más **oscura**.
- **`--ion-color-primary-tint`**: Una versión más **clara**.

Puedes usar el Generador de Colores de Ionic <https://ionicframework.com/docs/theming/color-generator> para que calcule todas estas variantes por ti.

Aplicando los Colores del Tema en el HTML

Una vez que has definido tu paleta de colores en `variables.scss`, usarlos en tu aplicación es muy sencillo. La mayoría de los componentes de Ionic tienen un atributo llamado **`color`** al que le puedes pasar el nombre del color que quieres aplicar.

Ejemplo de uso en `home.page.html`:

```html
<!-- En tu home.page.html -->
<ion-toolbar color="primary">...</ion-toolbar>
<ion-button color="secondary">...</ion-button>
<ion-button color="danger">...</ion-button>
```

Uso Avanzado: Aplicando `shade` y `tint` a Fondos

Aunque no es recomendable usar los colores primarios o secundarios para fondos grandes, sí puedes usar sus variantes `shade` y `tint` para crear efectos sutiles en elementos como tarjetas.

Paso A: Definir las clases en `global.scss`

```
/* Contenido para añadir en: src/global.scss */
.card-sombreada {
  --background: var(--ion-color-primary-shade);
  --color: var(--ion-color-primary-contrast);
}
```

Paso B: Usar la clase en el HTML

```
<!-- En tu home.page.html -->
<ion-item class="card-sombreada">...</ion-item>
```

Personalizando Variables Globales del Tema

Además de la paleta de colores, puedes controlar otros aspectos de la apariencia añadiendo estas variables a `:root`:

- **`--ion-background-color`**, **`--ion-text-color`**, etc.

Ejemplo de personalización global en `variables.scss`:

```
/* En src/theme/variables.scss, dentro de :root */
:root {
  /* ... tu paleta de colores ... */
  --ion-background-color: #f4f5f8;
  --ion-text-color: #222428;
}
```

Puedes encontrar la lista completa en la documentación oficial <https://ionicframework.com/docs/theming/themes> .

Jerarquía Visual con Colores Escalonados (Stepped Colors)

Para crear una sensación de profundidad, Ionic proporciona "colores escalonados". Debemos generarlos nosotros mismos y añadirlos a nuestro archivo `variables.scss`.

1. Define tu color de fondo base: `--ion-background-color: #f4f5f8;`.
2. Usa la herramienta de la documentación <https://ionicframework.com/docs/theming/themes#stepped-colors> para generar toda la escala de colores.
3. Pega el código generado en tu archivo `variables.scss` (dentro de `:root`).
4. Ahora ya puedes usar variables como `--ion-background-color-step-500`.

```
/* Contenido para añadir en: src/global.scss */
ion-item {
  --background: var(--ion-background-color-step-500);
}
```

Herramientas para Encontrar Paletas de Colores

Elegir una buena combinación de colores puede ser difícil. Afortunadamente, existen muchas herramientas online que te pueden ayudar a encontrar inspiración y a crear paletas armoniosas:

- Adobe Color CC <https://color.adobe.com/es/create/color-wheel> : Una herramienta muy completa para crear y explorar paletas basadas en reglas de armonía de color.
- Coolors <https://coolors.co/> : Muy popular por su interfaz sencilla para generar paletas aleatorias y ajustarlas.
- Color Hunt <https://colorhunt.co/> : Una colección curada de paletas de colores creadas por diseñadores. Ideal para encontrar inspiración rápida.
- Paletton <https://paletton.com/> : Similar a Adobe Color, te ayuda a encontrar colores complementarios, análogos, etc., a partir de un color base.
- COLOURlovers <https://www.colourlovers.com/> : Una comunidad donde la gente comparte paletas de colores, patrones y discute sobre tendencias.

✅ Práctica Rápida

Define una paleta de colores para tu proyecto personal. Elige un color primario y uno secundario que encajen con tu temática y aplícalos en el archivo `variables.scss`. Prueba

> a usar el color secundario en alguno de los botones de tu aplicación añadiendo el atributo `color="secondary"`.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.5. Modo Claro y Modo Oscuro (Dark Mode)

3.5. Modo Claro y Modo Oscuro (Dark Mode)

Ya tenemos un servicio capaz de guardar y leer datos del almacenamiento del dispositivo. Ahora vamos a utilizarlo para implementar una funcionalidad real: **recordar si el usuario prefiere el Modo Oscuro**.

Paso 1: Crear la Página de Ajustes

Primero necesitamos una interfaz donde el usuario pueda cambiar esta preferencia. Generamos una nueva página:

```
ionic generate page pages/ajustes
```

En el HTML de esta página (`src/app/pages/ajustes/ajustes.page.html`), vamos a aprovechar la estructura nativa que nos da Ionic, pero añadiendo el botón para volver atrás y nuestra lista de opciones.

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-buttons slot="start">
      <!-- Botón para volver a la Home -->
      <ion-back-button defaultHref="/home"></ion-back-button>
    </ion-buttons>
    <ion-title>Ajustes</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true">
```

```
    <!-- Header colapsable para iOS -->
    <ion-header collapse="condense">
      <ion-toolbar>
        <ion-title size="large">Ajustes</ion-title>
      </ion-toolbar>
    </ion-header>

    <!-- Nuestra lista de configuración -->
    <ion-list>
      <ion-list-header>Apariencia</ion-list-header>
      <ion-item>
        <ion-label>Modo Oscuro</ion-label>
        <!-- Enlazamos el toggle con una variable y escuchamos el cambio -->
        <ion-toggle [(ngModel)]="modoOscuro" (ionChange)="cambiarModoOscuro()"><
      </ion-item>
    </ion-list>
  </ion-content>
```

Paso 2: Conectar la Página con el Servicio

Ahora vamos a la lógica de la página (`src/app/pages/ajustes/ajustes.page.ts`). Aquí inyectamos nuestro `SettingsService` y programamos la lógica para guardar el cambio. Asegúrate de importar todos los componentes que usas en el HTML.

```
// src/app/pages/ajustes/ajustes.page.ts

import { Component, OnInit } from '@angular/core';
import { SettingsService } from 'src/app/services/settings.service';
import { FormsModule } from '@angular/forms';
import { IonContent, IonHeader, IonTitle, IonToolbar, IonList, IonListHeader,

@Component({
  selector: 'app-ajustes',
  templateUrl: './ajustes.page.html',
  styleUrls: ['./ajustes.page.scss'],
  standalone: true,
  imports: [
    IonContent, IonHeader, IonTitle, IonToolbar,
    IonButtons, IonBackButton,
    IonList, IonListHeader, IonItem, IonLabel, IonToggle,
    FormsModule
  ]
```

```
})
export class AjustesPage implements OnInit {

  modoOscuro: boolean = false; // Valor por defecto

  constructor(private settingsService: SettingsService) { }

  // ¡IMPORTANTE! Añadimos 'async' para poder usar 'await' dentro
  async ngOnInit() {
    // Al entrar, cargamos el valor guardado
    // Si no existe (es la primera vez), settingsService.get devuelve null,
    // así que usamos '|| false' para que sea false por defecto.
    this.modoOscuro = await this.settingsService.get('modo_oscuro') || false;

    // Aplicamos el tema inmediatamente al entrar por si acaso
    this.aplicarTema(this.modoOscuro);
  }

  // También debe ser async porque settingsService.set devuelve una promesa
  async cambiarModoOscuro() {
    // 1. Guardamos el nuevo valor en la base de datos
    await this.settingsService.set('modo_oscuro', this.modoOscuro);

    // 2. Aplicamos el cambio visualmente
    this.aplicarTema(this.modoOscuro);
  }

  aplicarTema(esOscuro: boolean) {
    // Añadimos o quitamos la clase 'dark' al body del documento
    // Esto activa los estilos que definiremos en variables.scss
    document.body.classList.toggle('dark', esOscuro);
  }
}
```

> ### Consejo Pro: Unificar Estilos con Mixins (SCSS)
>
> Para no tener que copiar y pegar los mismos colores dos veces (una para el modo automático y otra para el manual), podemos usar un **Mixin** de Sass en `variables.scss`.

```
/* src/theme/variables.scss */
```

```
/* 1. Definimos un "grupo" de estilos reutilizable (mixin) */
@mixin dark-theme-variables {
  --ion-color-primary: #428cff;
  --ion-color-primary-rgb: 66,140,255;
  --ion-color-primary-contrast: #ffffff;

  --ion-background-color: #121212;
  --ion-text-color: #ffffff;
  /* ... resto de variables ... */
}

/* 2. Lo aplicamos para detección automática (Sistema) */
@media (prefers-color-scheme: dark) {
  body {
    @include dark-theme-variables;
  }
}

/* 3. Lo aplicamos para activación manual (Clase .dark) */
body.dark {
  @include dark-theme-variables;
}
```

Con esto, defines los colores en un solo sitio y funcionan tanto si el usuario cambia el ajuste en su móvil como si pulsa el botón en tu app.

Paso 3: Cargar el Ajuste al Iniciar la App

Lo que hemos hecho hasta ahora funciona, pero tiene un fallo: si cierras la app y la vuelves a abrir, el modo oscuro no se aplicará hasta que visites la página de ajustes. ¡Queremos que se aplique nada más abrir la app!

Para eso, debemos ir al componente raíz de toda la aplicación: **`src/app/app.component.ts`**.

```
// src/app/app.component.ts

import { Component } from '@angular/core';
import { IonApp, IonRouterOutlet } from '@ionic/angular/standalone';
import { SettingsService } from './services/settings.service';
```

```
@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  imports: [IonApp, IonRouterOutlet],
})
export class AppComponent {

  constructor(private settingsService: SettingsService) {}

  // Angular ejecutará este método automáticamente al iniciar,
  // aunque no implementemos la interfaz OnInit explícitamente.
  // ¡IMPORTANTE! Añadimos 'async' para esperar a la base de datos.
  async ngOnInit() {
    await this.cargarPreferencias();
  }

  async cargarPreferencias() {
    // Leemos el ajuste guardado
    const modoOscuro = await this.settingsService.get('modo_oscuro');

    // Si era true, activamos el modo oscuro inmediatamente
    if (modoOscuro) {
      document.body.classList.add('dark');
    }
  }
}
```

> ### ¿Qué es `ngOnInit`?
>
> `ngOnInit` es un método especial del ciclo de vida de Angular. Angular lo llama automáticamente **una sola vez**, justo después de crear el componente y configurar sus propiedades. Aunque no es obligatorio poner `implements OnInit` en la clase, Angular buscará este método y lo ejecutará si existe.
>
> #### ¿Por qué no usar el `constructor`?
>
> El `constructor` debe usarse solo para la inyección de dependencias (pedir servicios). Si intentamos hacer lógica compleja o llamadas a bases de datos en el constructor, podríamos bloquear la creación del componente o encontrarnos con que la vista aún no

> está lista. Por eso, cualquier lógica de inicialización (como cargar datos) siempre debe ir en `ngOnInit`.

> ### Concepto Clave: `ngOnInit` Asíncrono
>
> Por defecto, el método `ngOnInit()` se genera sin la palabra clave `async`. Sin embargo, como necesitamos leer datos de la base de datos (una operación que tarda un tiempo), debemos usar `await` para esperar el resultado.
>
> **Recordatorio de la regla de oro:**
>
> - **`await` (esperar):** Detiene la ejecución del código en esa línea hasta que la promesa (la operación de base de datos) termine y devuelva el valor.
> - **`async` (asíncrono):** Para poder usar `await` dentro de una función, **es obligatorio** marcar esa función como `async`.
>
> Por eso hemos añadido la firma: `async ngOnInit() { ... }`.

Paso 4: Añadir Acceso desde la Home

Finalmente, necesitamos una forma de llegar a la página de ajustes. Vamos a añadir un botón con icono de engranaje en la barra de herramientas de `home.page.html`.

4.1. Importar Dependencias en `home.page.ts`

Para que el enlace funcione, asegúrate de importar `RouterLink`, `IonButtons` e `IonIcon`.

```
// src/app/home/home.page.ts
import { RouterLink } from '@angular/router';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonIn
import { addIcons } from 'ionicons';
import { settingsOutline } from 'ionicons/icons';

@Component({
  // ...
  imports: [
    // ... otros imports ...
    RouterLink,
    IonButtons, // <--- ¡Imprescindible!
```

```
    IonIcon     // <--- ¡Imprescindible!
  ]
})
export class HomePage {
  constructor(/*...*/) {
    // Registramos el icono de ajustes
    addIcons({ settingsOutline });
  }
  // ...
}
```

4.2. Añadir el Botón en `home.page.html`

Asegúrate de que `<ion-buttons>` esté fuera de `<ion-title>`. Deben ser "hermanos".

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-title>Mis Tareas</ion-title>

    <!-- CORRECTO: ion-buttons está fuera de ion-title -->
    <ion-buttons slot="end">
      <ion-button routerLink="/ajustes">
        <ion-icon slot="icon-only" name="settings-outline"></ion-icon>
      </ion-button>
    </ion-buttons>

  </ion-toolbar>
</ion-header>
```

¡Y listo! Ahora tu aplicación tiene memoria y un sistema de preferencias completo. Da igual cuántas veces la cierres o reinicies; si el usuario eligió el Modo Oscuro, Ionic Storage lo recordará y tu `AppComponent` lo aplicará al instante.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.6. Usando Iconos con Ionicons

3.6. Usando Iconos con Ionicons

Una interfaz de usuario profesional no solo se compone de texto y botones, sino también de **iconos** que ayudan a comunicar acciones y conceptos de forma rápida y visual. Ionic viene con su propia librería de iconos de alta calidad, llamada **Ionicons**.

En esta sección, aprenderás a buscar, importar y utilizar estos iconos en tu aplicación.

¿Por qué los iconos no aparecen por defecto?

Al igual que con los componentes de Ionic en un proyecto `standalone`, los iconos no se incluyen todos por defecto para optimizar el tamaño final de la aplicación. Ionicons tiene cientos de iconos, y cargarlos todos haría que la app pesara mucho. Por ello, debemos **importar explícitamente** solo aquellos que vayamos a necesitar.

El Proceso de 3 Pasos para Usar un Icono

El mejor lugar para registrar los iconos que usarás en toda tu aplicación es en el archivo **`src/app/app.component.ts`**.

1. **Buscar el icono:** Ve a la web oficial de Ionicons <https://ionic.io/ionicons> y busca el icono que necesitas. Al hacer clic en él, te mostrará su nombre para el HTML (ej: `"trash"`) y su nombre para la importación en TypeScript (ej: `trash`).
2. **Importar el icono:** En tu archivo `app.component.ts`, importa la función `addIcons` y el icono específico que has encontrado.
3. **Registrar el icono:** En el constructor de la clase, llama a la función `addIcons` pasándole un objeto con los iconos que has importado.

Ejemplo Práctico: Añadiendo Iconos de Papelera y Edición

Vamos a registrar los iconos para las acciones de "eliminar" (trash) y "editar" (create).

```
// Contenido para añadir en: src/app/app.component.ts

import { Component } from '@angular/core';
import { IonicModule } from '@ionic/angular';

// 1. Importa la función y los iconos que necesites
import { addIcons } from 'ionicons';
import { trash, create } from 'ionicons/icons';

@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  styleUrls: ['app.component.scss'],
  standalone: true,
  imports: [IonicModule],
})
export class AppComponent {
  constructor() {
    // 2. Llama a addIcons en el constructor con un objeto de los iconos
    addIcons({ trash, create });
  }
}
```

Paso Final: Usar el Icono en el HTML

Una vez que el icono está registrado, ya puedes usarlo en cualquier página de tu aplicación con la etiqueta **`<ion-icon>`** y su atributo `name`.

```
<!-- En cualquier archivo .html de tu proyecto -->

<ion-button color="danger">
  <ion-icon slot="start" name="trash"></ion-icon>
  Eliminar
</ion-button>

<ion-button color="secondary">
  <ion-icon slot="start" name="create"></ion-icon>
  Editar
</ion-button>
```

¿Qué es el atributo `slot`?

El atributo **`slot`** es una característica de los Web Components (la tecnología base de los componentes de Ionic) que le dice al componente padre *dónde* debe colocar un elemento hijo. Piensa en ello como si el componente padre (`<ion-button>`) tuviera diferentes "ranuras" o "huecos" con nombre donde puedes enchufar contenido.

Los valores más comunes que usarás son:

- **`slot="start"`**: Coloca el elemento al principio del componente padre (a la izquierda en un botón o en un `ion-item`).
- **`slot="end"`**: Coloca el elemento al final del componente padre (a la derecha).
- **`slot="icon-only"`**: Se usa en un `<ion-button>` para indicar que el botón solo contendrá un icono, sin texto, y Ionic le aplicará los estilos adecuados (forma redonda, etc.).

En nuestro ejemplo, `slot="start"` le dice al botón que coloque el icono antes del texto "Eliminar".

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.7. Tipografía y Fuentes Personalizadas

3.7. Tipografía y Fuentes Personalizadas

La tipografía es uno de los elementos de diseño más importantes para definir la personalidad de una aplicación. Por defecto, Ionic utiliza las fuentes del sistema operativo (San Francisco en iOS, Roboto en Android), lo que garantiza una apariencia nativa y un rendimiento óptimo.

Sin embargo, a veces queremos usar una fuente personalizada para que nuestra marca tenga una identidad visual única. En esta sección, aprenderás a integrar una fuente de **Google Fonts** en tu proyecto.

Paso 1: Elegir una Fuente en Google Fonts

1. Ve a Google Fonts <https://fonts.google.com/> y busca la tipografía que más te guste. Para nuestro ejemplo temático de Halloween, usaremos **"Henny Penny"**.
2. Selecciona el estilo que necesites. En el panel lateral que aparece, busca la opción **`@import`** y copia la URL que te proporciona.

Paso 2: Importar la Fuente en tu Aplicación

El mejor lugar para importar una fuente que se usará en toda la aplicación es el archivo de estilos global: **`src/global.scss`**.

1. Abre el archivo `src/global.scss`.
2. Pega la línea `@import` que copiaste de Google Fonts al principio del todo.

```
/* Contenido para añadir en: src/global.scss */

/* Importamos la fuente Henny Penny */
@import url('https://fonts.googleapis.com/css2?family=Henny+Penny&display=swap
```

```
/* El resto de estilos globales de tu aplicación irían aquí debajo */
```

> ### ¿Qué es el parámetro `&display=swap`?
>
> Este parámetro es una optimización de rendimiento muy importante. Le dice al navegador que muestre el texto inmediatamente con una fuente de respaldo (fallback) mientras la fuente personalizada se descarga en segundo plano. Una vez cargada, se intercambian ("swap"). Esto evita que el usuario vea texto invisible mientras espera, mejorando la experiencia de carga de la aplicación.

Paso 3: Aplicar la Fuente a toda la Aplicación

Ahora que la fuente ya está "disponible" en nuestra app, tenemos que decirle a Ionic que la use. Para ello, volvemos a nuestro archivo de theming, **`src/theme/variables.scss`**, y sobreescribimos la variable CSS que controla la tipografía: **`--ion-font-family`**.

1. Abre el archivo `src/theme/variables.scss`.
2. Dentro del selector `:root` (donde definimos los colores), añade la siguiente línea:

```
/* Contenido para añadir en: src/theme/variables.scss */

:root {

  /* ... tus variables de color ... */

  /** Font Family **/
  --ion-font-family: 'Henny Penny', cursive;
}
```

¡Y ya está! Si guardas los cambios, verás que toda la tipografía de tu aplicación (títulos, párrafos, botones...) ha cambiado automáticamente a la nueva y terrorífica fuente "Henny Penny".

La parte `, cursive` se añade como un "fallback": si por cualquier motivo la fuente Henny Penny no se pudiera cargar, el navegador usaría una fuente genérica de estilo manuscrito en su lugar.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.8. Mostrando Imágenes en tu Aplicación

3.8. Mostrando Imágenes en tu Aplicación

Las imágenes son una parte vital de cualquier interfaz de usuario. En Ionic, tenemos principalmente dos formas de incluirlas en nuestra aplicación: como archivos locales que forman parte del proyecto (assets) o como imágenes remotas cargadas desde una URL.

Método 1: Imágenes Locales (Assets)

Este es el método que usarás para las imágenes que son una parte integral de tu aplicación, como logos, iconos personalizados o imágenes de fondo. Estos archivos se "empaquetan" junto con tu app.

Paso 1: Organizar las imágenes en la carpeta `assets`

Dentro de la carpeta `src/` de tu proyecto, encontrarás una carpeta llamada **`assets`**. Este es el lugar designado para todos tus recursos estáticos (imágenes, fuentes, archivos JSON, etc.).

Es una buena práctica crear subcarpetas dentro de `assets` para mantener todo organizado. Por ejemplo, vamos a crear una carpeta `img` para nuestras imágenes.

Estructura de carpetas:

```
src/
└── assets/
    └── img/
        └── mi-logo.png
```

Paso 2: Mostrar la imagen en el HTML

Para hacer referencia a una imagen que está en la carpeta `assets`, usamos una ruta relativa que siempre empieza con `assets/`. Ionic sabe automáticamente dónde encontrar esa carpeta.

```
<!-- En cualquier archivo .html de tu proyecto -->
<img src="assets/img/mi-logo.png" alt="Logo de mi aplicación">
```

Método 2: Imágenes Remotas (desde una URL)

Este método es el que usarás cuando las imágenes provengan de una fuente externa, como una API o cualquier otro sitio web. Las imágenes no se guardan en tu proyecto, sino que se cargan desde internet cada vez que se necesitan.

Para ello, simplemente tienes que usar la URL completa de la imagen en el atributo `src`.

```
<!-- Usando una URL completa -->
<img src="https://ionicframework.com/docs/img/logo.svg" alt="Logo de Ionic">
```

El Componente `<ion-img>`: La Alternativa Optimizada

Aunque puedes usar la etiqueta estándar `<img>` sin problemas, Ionic nos proporciona su propio componente, **`<ion-img>`**, que ofrece una ventaja muy importante: la **carga perezosa (Lazy Loading)**.

> ### ¿Qué es el Lazy Loading?
>
> Imagina una página con 50 imágenes. Con una etiqueta `<img>` normal, el navegador intentaría descargar las 50 imágenes a la vez, lo que haría que la página cargara muy lenta.
>
> Con `<ion-img>`, la imagen solo se descarga cuando está a punto de aparecer en la pantalla del usuario (por ejemplo, cuando el usuario hace scroll hacia ella). Esto mejora enormemente el rendimiento y la velocidad de carga inicial de tus páginas.

Su uso es idéntico al de una etiqueta `<img>`. Simplemente, reemplaza la etiqueta.

```
<!-- Usando ion-img con una imagen local -->
```

```
<ion-img src="assets/img/mi-logo.png" alt="Logo de mi aplicación"></ion-img>

<!-- Usando ion-img con una imagen remota -->
<ion-img src="https://ionicframework.com/docs/img/logo.svg" alt="Logo de Ionic
```

Recomendación: A partir de ahora, acostúmbrate a usar siempre **`<ion-img>`** en lugar de `<img>` para todas las imágenes de tu aplicación. No olvides importarlo en el array `imports` de tus componentes `standalone`.

Otros Componentes para Imágenes: Avatares y Miniaturas

Además de `<ion-img>`, Ionic nos ofrece dos componentes contenedores especializados para casos de uso muy comunes: `<ion-avatar>` y `<ion-thumbnail>`.

`<ion-avatar>`: Para Imágenes de Perfil

Este componente es un contenedor que aplica automáticamente los estilos necesarios para que una imagen se vea como un avatar (generalmente, de forma circular). Es ideal para listas de usuarios, comentarios o perfiles.

```
<ion-item>
  <!-- El avatar se coloca al principio del item -->
  <ion-avatar slot="start">
    <ion-img src="https://i.pravatar.cc/150?u=a042581f4e29026024d"></ion-img>
  </ion-avatar>
  <ion-label>
    <h3>Nombre de Usuario</h3>
    <p>Última conexión: hace 5 min</p>
  </ion-label>
</ion-item>
```

`<ion-thumbnail>`: Para Miniaturas en Listas

Este componente es similar a `<ion-avatar>`, pero está pensado para mostrar imágenes en miniatura (thumbnails) en listas de elementos, como productos, noticias o canciones. Por defecto, aplica una forma cuadrada.

```
<ion-item>
  <!-- La miniatura se coloca al principio del item -->
  <ion-thumbnail slot="start">
    <ion-img src="https://placehold.co/100x100/3dc2ff/ffffff?text=Producto"></
  </ion-thumbnail>
  <ion-label>
    <h3>Nombre del Producto</h3>
    <p>Categoría del producto</p>
  </ion-label>
</ion-item>
```

Usar estos contenedores no solo te ahorra escribir CSS, sino que también hace que tu código sea más semántico y fácil de leer, ya que queda claro cuál es el propósito de cada imagen.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.9. Recogiendo Datos del Usuario con Formularios

3.9. Recogiendo Datos del Usuario con Formularios

Hasta ahora, nuestra aplicación de lista de tareas solo muestra datos fijos. ¡Es hora de hacerla interactiva! Los **formularios** son la clave para permitir al usuario añadir sus propias tareas.

En esta sección, aprenderemos a usar los componentes de formulario básicos de Ionic y a conectarlos con la lógica de nuestra aplicación usando el "two-way data binding": **`[(ngModel)]`**.

> #### El Único Requisito para que `[(ngModel)]` Funcione
>
> Para que la directiva `[(ngModel)]` funcione correctamente en un entorno `standalone`, solo hay un requisito que siempre debes cumplir:
>
> 1. **Importar** **`FormsModule`** en el array `imports` de tu componente.
>
> Si alguna vez `[(ngModel)]` te da un error, ¡asegúrate de que has importado este módulo!

Componentes de Formulario Esenciales en Ionic

Ionic nos proporciona una completa librería de componentes para construir formularios ricos y funcionales. Los más importantes son:

- **`<ion-input>`**: Para campos de texto de una sola línea (texto, email, contraseña, número...).
- **`<ion-textarea>`**: Para campos de texto de múltiples líneas.
- **`<ion-checkbox>`**: Una casilla de verificación para opciones individuales.
- **`<ion-toggle>`**: El clásico interruptor de "encendido/apagado".

- **`<ion-radio>` y `<ion-radio-group>`**: Para que el usuario elija una única opción de un grupo.
- **`<ion-select>`**: Para crear listas desplegables donde el usuario puede elegir una o varias opciones.
- **`<ion-range>`**: Un control deslizante (slider) para seleccionar un valor numérico dentro de un rango.

Organizando el Formulario con `<ion-list>`

Aunque en la UT2 usamos `<ion-item>` para mostrar cada tarea, no introdujimos formalmente su contenedor más común: **`<ion-list>`**. Esta etiqueta es el contenedor perfecto para agrupar una serie de `<ion-item>`, ya que aplica estilos de forma automática (como líneas separadoras) y mejora la estructura semántica de nuestro código. Es la práctica recomendada para construir formularios y listas en Ionic.

> ### Consulta la Documentación Oficial
>
> Cada uno de estos componentes de formulario tiene muchísimos atributos (`properties`), eventos (`events`) y partes estilizables (`CSS custom properties`) que no podemos cubrir en detalle aquí. La documentación oficial de componentes de Ionic <https://ionicframework.com/docs/components> es tu mejor amiga para descubrirlos todos.

Ejemplo Práctico: Añadir Nuevas Tareas

Vamos a añadir un pequeño formulario en nuestra `home.page.html` para que el usuario pueda escribir el título de una nueva tarea y añadirla a la lista. En lugar de usar una variable simple para el título, usaremos un objeto completo de tipo `Tarea`.

1. La Lógica en `home.page.ts`

Preparamos el archivo TypeScript. Importamos `FormsModule` y los componentes necesarios. Declaramos una nueva propiedad `nuevaTarea` de tipo `Tarea` y la inicializamos. Modificamos el método `agregarTarea` para que trabaje con este objeto.

```
// Contenido para añadir/modificar en: src/app/home/home.page.ts

import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonInp
```

```
import { Tarea } from '../interfaces/tarea';
import { TaskItemComponent } from '../components/task-item/task-item.component
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  imports: [
    CommonModule, FormsModule, IonHeader, IonToolbar, IonTitle, IonContent,
    IonList, IonItem, IonInput, IonButton, TaskItemComponent
  ],
})
export class HomePage {

  listaDeTareas: Tarea[] = [
    { id: 1, titulo: "Comprar el pan", completada: true },
    { id: 2, titulo: "Estudiar componentes de Angular", completada: false }
  ];

  // Nueva propiedad: un objeto Tarea para enlazar con el formulario
  public nuevaTarea: Tarea = {
    id: 0, // Daremos un ID real al añadirla
    titulo: "", // Enlazaremos esto al input
    completada: false // Valor por defecto
  };

  constructor() {}

  // Método modificado para añadir la tarea desde el objeto
  agregarTarea() {
    // Verificamos que el título no esté vacío
    if (this.nuevaTarea.titulo.trim().length === 0) {
      return; // No hacemos nada si está vacío
    }

    // Creamos una copia del objeto para añadir a la lista
    // Le asignamos un ID único y usamos el título del formulario
    const tareaParaAnadir: Tarea = {
      ...this.nuevaTarea, // Copia las propiedades existentes (titulo, complet
      id: Date.now()     // Asigna un nuevo ID
    };

    // Añadimos la nueva tarea al principio del array
    this.listaDeTareas.unshift(tareaParaAnadir);
```

```
    // Reseteamos el objeto del formulario para la siguiente tarea
    this.nuevaTarea = { id: 0, titulo: "", completada: false };
  }
}
```

Desglosando el Método `agregarTarea()`

Este método contiene dos detalles importantes:

- **`...this.nuevaTarea` (Spread Syntax):** El operador `...` se llama "sintaxis de propagación". Lo que hace es copiar todas las propiedades del objeto `this.nuevaTarea` (en este caso, `titulo` y `completada`) y "pegarlas" dentro del nuevo objeto `tareaParaAnadir`. Es una forma corta y moderna de crear una copia de un objeto.
- **`id: Date.now()`:** Cada tarea necesita un identificador único. `Date.now()` es una función de JavaScript que devuelve el número de milisegundos transcurridos desde el 1 de enero de 1970. Usarlo como ID es una forma muy sencilla de obtener un número único para cada tarea que creamos en esta sesión. **Importante:** Esta técnica es útil para ejemplos, pero no es ideal para aplicaciones reales donde los IDs deben ser más robustos y persistentes.

2. La Vista en `home.page.html`

Ahora, en la plantilla, enlazamos el `[(ngModel)]` del `<ion-input>` a la propiedad `titulo` de nuestro nuevo objeto `nuevaTarea`.

```
<!-- Contenido para añadir en: src/app/home/home.page.html (dentro de ion-cont

<!-- FORMULARIO PARA AÑADIR TAREAS -->
<ion-list>
  <ion-item>
    <ion-input
      label="Nueva Tarea"
      label-placement="floating"
      placeholder="¿Qué necesitas hacer?"
      [(ngModel)]="nuevaTarea.titulo"> <!-- Enlazado a la propiedad 'titulo' d
    </ion-input>
```

```
    </ion-item>
  </ion-list>

  <ion-button expand="block" (click)="agregarTarea()" class="ion-margin-top ion-
    Añadir Tarea
  </ion-button>

  <!-- LISTA DE TAREAS (El código que ya teníamos) -->
  <ion-grid>
    <ion-row>
      <ion-col *ngFor="let tarea of listaDeTareas" size="12" size-md="6" size-lg
        <app-task-item [tarea]="tarea"></app-task-item>
      </ion-col>
    </ion-row>
  </ion-grid>
```

¡Y listo! Aunque visualmente el resultado es el mismo, ahora estamos usando una técnica mucho más escalable. Si en el futuro quisiéramos añadir más campos al formulario (como una fecha límite o una descripción), solo tendríamos que añadir las propiedades correspondientes al objeto `nuevaTarea` y los nuevos `<ion-input>` o `<ion-textarea>` enlazados a ellas.

> ### Nota sobre Modales
>
> Para formularios más complejos (con múltiples campos, validaciones, etc.), es una práctica muy común presentarlos dentro de una **ventana modal** en lugar de directamente en la página. Aprenderemos a crear y gestionar modales en la siguiente sección sobre "Overlays".

> ### Personalizando el Color de la Etiqueta Flotante
>
> Como has observado, cuando un `<ion-input>` con `label-placement="floating"` recibe el foco, la etiqueta flotante adopta el color primario de tu aplicación. Si quieres cambiar este comportamiento, puedes hacerlo con CSS.
>
> Añade la siguiente regla en el archivo **`home.page.scss`** (o en `global.scss` si quieres que afecte a todos los inputs de la app):

```scss
/* Contenido para añadir en: src/app/home/home.page.scss */

/* Cambia el color de la etiqueta flotante cuando el input está enfocado
ion-input.has-focus .label-text {
  color: var(--ion-color-secondary); /* Usa el color secundario, por ejem
}

/* Opcional: También puedes cambiar el color de la línea inferior */
ion-input.has-focus {
  --highlight-color-focused: var(--ion-color-secondary);
}
```

Con esta regla, le estamos diciendo a Ionic: "Cuando un `ion-input` tenga la clase `.has-focus` (que se añade automáticamente al hacer clic), busca el elemento con la clase `.label-text` dentro de él y aplícale el color secundario". ¡Experimenta con diferentes colores o variables!

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.10. Notificaciones y Ventanas Emergentes (Overlays)

3.10. Notificaciones y Ventanas Emergentes (Overlays)

Una aplicación no solo debe mostrar información, también necesita comunicarse activamente con el usuario para darle feedback, pedirle confirmaciones o solicitarle datos más complejos. Para ello, Ionic nos proporciona una serie de componentes llamados **"Overlays"** (superposiciones), que aparecen por encima del contenido principal de la aplicación.

En esta sección, veremos los tres más importantes: `ion-toast`, `ion-alert` y `ion-modal`.

El Patrón "Controller"

A diferencia de los componentes que hemos visto hasta ahora, los overlays no se escriben directamente en el archivo `.html`. Se crean y se controlan desde la lógica, en nuestro archivo `.ts`, utilizando un patrón de diseño llamado "Controller".

El flujo es siempre el mismo:

1. **Importar** el controlador correspondiente (ej: `ToastController`) en nuestro archivo `.ts`.
2. **Inyectarlo** en el `constructor` de nuestra clase para poder usarlo.
3. Crear un método **`async`** que use el controlador para `create()` y `present()` el overlay.

> #### Concepto Clave 1: Inyección de Dependencias en el Constructor
>
> La **Inyección de Dependencias** es un mecanismo de Angular donde el `constructor` le pide a Angular las "piezas" (servicios como `ToastController`) que necesita para funcionar. La palabra clave `private` es un atajo que pide y guarda la pieza automáticamente como una propiedad de la clase (`this.toastController`).

> ### Concepto Clave 2: Métodos Asíncronos (`async / await`)
>
> Mostrar un overlay es una **operación asíncrona** (no es instantánea). `async / await` es una forma de escribir código asíncrono que se lee como si fuera síncrono. `async` marca el método, y `await` pausa la ejecución en una línea hasta que una operación (como `create()` o `present()`) se completa.

Tipos de Overlays

- **`ion-toast`**: Mensajes breves y no intrusivos (ej: "Guardado").
- **`ion-alert`**: Ventanas de diálogo para alertas o confirmaciones (ej: "¿Seguro?").
- **`ion-modal`**: Ventanas completas para contenido complejo o formularios.

Ejemplo Práctico: Implementando Overlays

Vamos a ver cómo implementar cada uno de ellos paso a paso en nuestra `home.page.ts`.

Paso 0: Configuración Base (Imports e Inyección)

Primero, preparamos nuestro componente importando los controladores e inyectándolos en el constructor. Esta es la base sobre la que construiremos el resto.

```
// src/app/home/home.page.ts
import { Component } from '@angular/core';
// 1. Importamos los controladores desde 'standalone'
import { ToastController, AlertController, ModalController } from '@ionic/angu
import { IonHeader, IonToolbar, IonTitle, IonContent, IonButton } from '@ionic
// Importamos el componente del modal (lo crearemos más adelante)
import { DetalleModalComponent } from '../components/detalle-modal/detalle-moc

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  imports: [IonHeader, IonToolbar, IonTitle, IonContent, IonButton],
})
export class HomePage {
```

```
  // 2. Inyectamos los tres controladores en el constructor
  constructor(
    private toastController: ToastController,
    private alertController: AlertController,
    private modalController: ModalController
  ) {}

  // ... Aquí añadiremos los métodos a continuación ...
}
```

Paso 1: Implementando Toasts

Añade este método dentro de la clase `HomePage`. Un Toast es perfecto para avisos rápidos.

```
async mostrarToast() {
  const toast = await this.toastController.create({
    message: '¡Hola! Esto es una notificación Toast.',
    duration: 2000, // Se cierra solo a los 2 segundos
    position: 'bottom', // 'top', 'middle', 'bottom'
    color: 'success',   // Color del tema (opcional)
    buttons: ['OK']     // Botón de cierre opcional
  });
  await toast.present();
}
```

Paso 2: Implementando Alertas

Las alertas pueden ser simples (solo mensaje) o tener botones con acciones (confirmación).

```
// Alerta informativa simple
async mostrarAlertaSimple() {
  const alert = await this.alertController.create({
    header: 'Información',
    subHeader: 'Mensaje importante',
    message: 'Esto es una alerta que requiere atención.',
    buttons: ['Entendido']
  });
  await alert.present();
}
```

```
  // Alerta de confirmación con lógica
  async mostrarAlertaConfirmacion() {
    const alert = await this.alertController.create({
      header: '¿Estás seguro?',
      message: 'Esta acción no se puede deshacer.',
      buttons: [
        {
          text: 'Cancelar',
          role: 'cancel', // Cierra la alerta sin más
          handler: () => { console.log('Cancelado'); }
        },
        {
          text: 'Eliminar',
          role: 'confirm',
          handler: () => {
            console.log('Confirmado: Elemento eliminado');
            // Aquí llamaríamos a la lógica de borrado
          }
        }
      ]
    });
    await alert.present();
  }
```

Paso 3: Implementando Modales (Comunicación Bidireccional)

Los modales requieren un poco más de trabajo porque necesitan un componente propio.

3.1 Crear y Configurar el Componente del Modal

Generamos el componente: `ionic g c components/detalle-modal`. Luego editamos su lógica para recibir datos (`@Input`) y cerrar devolviendo datos.

```
// src/app/components/detalle-modal/detalle-modal.component.ts
import { Component, Input } from '@angular/core';
// ¡OJO! Importar ModalController desde 'standalone'
import { ModalController, IonHeader, IonToolbar, IonTitle, IonButtons, IonButt

@Component({
  /* ... selectores y template ... */
  standalone: true,
  imports: [IonHeader, IonToolbar, IonTitle, IonButtons, IonButton, IonContent
```

```
})
export class DetalleModalComponent {
  @Input() mensaje: string = ''; // Dato que recibimos

  constructor(private modalController: ModalController) {}

  cerrar() {
    // Cerramos devolviendo un dato
    this.modalController.dismiss({ leido: true }, 'confirm');
  }
}
```

3.2 Abrir el Modal desde la Home

Volvemos a `home.page.ts` y añadimos el método para abrirlo.

```
  async abrirModal() {
    const modal = await this.modalController.create({
      component: DetalleModalComponent,
      // Pasamos datos al modal
      componentProps: {
        mensaje: 'Este mensaje viene de la Home'
      }
    });

    await modal.present();

    // Esperamos a que se cierre para leer la respuesta
    const { data, role } = await modal.onDidDismiss();

    if (role === 'confirm') {
      console.log('Modal cerrado. Datos recibidos:', data);
    }
  }
```

Paso 4: La Vista (Botones)

Finalmente, conectamos todo en el HTML.

```
<!-- src/app/home/home.page.html -->

<ion-button expand="block" (click)="mostrarToast()">Toast</ion-button>
<ion-button expand="block" (click)="mostrarAlertaSimple()" color="secondary">A
<ion-button expand="block" (click)="mostrarAlertaConfirmacion()" color="danger
<ion-button expand="block" (click)="abrirModal()" color="warning">Abrir Modal<
```

Resolución de Problemas: Error "No provider for ModalController"

Si obtienes el error `NullInjectorError: No provider for ModalController!`, revisa tus importaciones. Es vital que importes los controladores desde el paquete **standalone**:

❌ **INCORRECTO (Causa el error):**

```
import { ModalController } from '@ionic/angular';
```

✅ **CORRECTO:**

```
import { ModalController } from '@ionic/angular/standalone';
```

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.11. Indicadores de Carga con Skeletons

3.11. Indicadores de Carga con Skeletons

Cuando nuestra aplicación necesita cargar datos desde una fuente externa (como una API, que veremos en la UT5), el proceso no es instantáneo. Durante unos instantes, la aplicación está esperando la respuesta y no tiene nada que mostrar. En lugar de dejar la pantalla en blanco o mostrar un simple "cargando...", las aplicaciones modernas usan una técnica de UX mucho más elegante: los **Textos de Esqueleto (Skeleton Text)**.

Un "skeleton" es una versión "fantasma" de tu interfaz que se muestra mientras los datos reales se están cargando. Da al usuario una idea inmediata de cómo será el contenido, lo que reduce la percepción del tiempo de espera y hace que la aplicación se sienta más rápida y profesional.

El Componente `<ion-skeleton-text>`

Ionic nos lo pone muy fácil con el componente `<ion-skeleton-text>`. Este componente muestra una forma gris y animada que podemos adaptar para que tenga la misma apariencia que el contenido que va a reemplazar.

La clave para usarlo es crear una estructura HTML que sea una réplica exacta de tu componente real, pero reemplazando los elementos de contenido (imágenes, títulos, párrafos) por `<ion-skeleton-text>`.

Ejemplo Práctico: Un Skeleton para nuestro `TaskItemComponent`

Vamos a simular un estado de carga en nuestra aplicación de lista de tareas. Haremos que al entrar en la página se muestren 3 "tareas fantasma" durante unos segundos, y luego aparezcan las tareas reales.

1. La Lógica en `home.page.ts`

Necesitamos una nueva propiedad (un booleano) que controle si estamos en estado de "cargando" o no. Además, es crucial importar `CommonModule` para que funcionen las directivas

`*ngIf` y `*ngFor` que usaremos en la vista.

```
// Contenido para añadir/modificar en: src/app/home/home.page.ts

import { Component } from '@angular/core';
// 1. Importamos CommonModule (necesario para *ngIf y *ngFor)
import { CommonModule } from '@angular/common';
import { IonHeader, IonToolbar, IonTitle, IonContent } from '@ionic/angular/st
// 2. Importamos IonSkeletonText y los componentes de lista necesarios
import { IonList, IonItem, IonLabel, IonCheckbox, IonSkeletonText } from '@ior
import { Tarea } from '../interfaces/tarea';
import { TaskItemComponent } from '../components/task-item/task-item.component

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  // 3. Añadimos CommonModule y el resto de componentes al array de imports
  imports: [
    CommonModule, // ¡Fundamental para que *ngIf y *ngFor funcionen!
    IonHeader, IonToolbar, IonTitle, IonContent,
    IonList, IonItem, IonLabel, IonCheckbox, IonSkeletonText, // Componentes p
    TaskItemComponent
  ],
})
export class HomePage {
  public listaDeTareas: Tarea[] = []; // Empezamos con la lista vacía
  public cargando: boolean = true; // Nueva propiedad para controlar el estado

  constructor() {
    // Simulamos una carga de datos de 2 segundos
    setTimeout(() => {
      this.listaDeTareas = [
        { id: 1, titulo: "Comprar el pan", completada: true },
        { id: 2, titulo: "Estudiar componentes de Angular", completada: false
        { id: 3, titulo: "Hacer el reto de la UT2", completada: false }
      ];
      this.cargando = false; // Cambiamos el estado a "cargado"
    }, 2000);
  }
}
```

> ### ¿Por qué hace falta importar `CommonModule`?
>
> En los componentes `standalone`, Angular no incluye nada "por arte de magia". Las directivas estructurales como **`*ngIf`** y **`*ngFor`** no están disponibles por defecto; pertenecen a un paquete de Angular llamado **`CommonModule`**.
>
> Si intentas usarlas en tu HTML sin haber importado `CommonModule` en el array `imports` de tu componente, obtendrás errores como *"NG8003: No directive found with exportAs 'ngIf'"*. Por eso, siempre que uses lógica condicional o bucles en tu vista, ¡recuerda importar este módulo!

2. La Vista en `home.page.html`

Ahora, usamos `*ngIf` para mostrar el esqueleto o la lista real, dependiendo del valor de nuestra variable `cargando`.

```
<!-- Contenido para añadir/modificar en: src/app/home/home.page.html -->
<ion-content [fullscreen]="true">
  <!-- ... cabeceras ... -->

  <!-- SECCIÓN DE CARGA (SKELETON) -->
  <!-- Este bloque solo se muestra mientras 'cargando' es true -->
  <div *ngIf="cargando">
    <ion-list>
      <!-- Repetimos la estructura del esqueleto 3 veces -->
      <ion-item *ngFor="let i of [1, 2, 3]">
        <ion-checkbox slot="start" disabled="true"></ion-checkbox>
        <ion-label>
          <h3>
            <!-- El skeleton-text reemplaza al título real -->
            <ion-skeleton-text [animated]="true" style="width: 80%;"></ion-ske
          </h3>
        </ion-label>
      </ion-item>
    </ion-list>
  </div>

  <!-- SECCIÓN DE CONTENIDO REAL -->
  <!-- Este bloque se muestra cuando la carga ha terminado -->
  <div *ngIf="!cargando">
```

```
      <div *ngIf="listaDeTareas.length === 0" class="mensaje-vacio">
        <p>¡Enhorabuena! No tienes tareas pendientes.</p>
      </div>

      <app-task-item
        *ngFor="let tarea of listaDeTareas"
        [tarea]="tarea">
      </app-task-item>
    </div>

  </ion-content>
```

Si ahora ejecutas la aplicación, verás el efecto: durante dos segundos, la pantalla mostrará las barras grises animadas, y justo después, aparecerá el contenido real. ¡Es una mejora de UX muy significativa con muy poco esfuerzo!

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.12. Interacciones Avanzadas con Gestos Táctiles

3.12. Interacciones Avanzadas con Gestos Táctiles

Las aplicaciones móviles van más allá de los simples clics. Los usuarios esperan poder interactuar con la interfaz de forma más directa y natural a través de **gestos táctiles** como deslizar, pellizcar para hacer zoom o mantener pulsado.

Ionic nos proporciona un potente **Controlador de Gestos (`GestureController`)** que nos permite capturar y reaccionar a estas interacciones complejas de una manera sorprendentemente sencilla.

En esta sección, aprenderemos a implementar uno de los gestos más comunes y útiles: el "deslizar para eliminar" (swipe to delete) en un elemento de nuestra lista de tareas.

El Patrón del Controlador de Gestos

Al igual que con los Overlays, la creación de gestos se gestiona desde la lógica de nuestro componente (el archivo `.ts`). El flujo es el siguiente:

1. **Importar** `GestureController` y las interfaces necesarias.
2. **Inyectar** el `GestureController` en el `constructor`.
3. Usar el "hook del ciclo de vida" **`ngAfterViewInit`** para asegurarnos de que la vista HTML ya está cargada antes de asociar el gesto.
4. Obtener una referencia al elemento HTML al que queremos aplicar el gesto.
5. Crear y configurar el gesto, definiendo qué debe pasar cuando empieza (`onStart`), cuando el dedo se mueve (`onMove`) y cuando se levanta (`onEnd`).

> **Conceptos Clave: `@ViewChild` y `ngAfterViewInit`**

- **`@ViewChild`**: Es un decorador de Angular que nos permite obtener una referencia a un elemento de nuestra plantilla HTML directamente en nuestro código TypeScript. Es como un `document.getElementById`, pero integrado en Angular.
- **`ngAfterViewInit()`**: Es otro "hook del ciclo de vida" de Angular. Se ejecuta automáticamente justo después de que la vista (el HTML) del componente se haya renderizado por completo. Es el lugar perfecto para manipular el DOM o, como en este caso, para asociar un gesto a un elemento que ya sabemos que existe.

Ejemplo Práctico: Deslizar para Eliminar en `TaskItemComponent`

Vamos a modificar nuestro componente de tarea para que, al deslizarlo hacia la izquierda, se prepare para ser eliminado.

1. La Vista en `task-item.component.html`

Primero, necesitamos una forma de identificar nuestra tarjeta en el código. Lo hacemos añadiendo una "variable de referencia de plantilla" (`#card`).

```
<!-- Archivo: src/app/components/task-item/task-item.component.html -->

<!-- Añadimos #card para poder referenciar este elemento desde el archivo .ts
<ion-card #card [color]="...">
  <!-- ... el resto del contenido del item ... -->
</ion-card>
```

2. La Lógica en `task-item.component.ts`

Aquí es donde ocurre la magia. Implementaremos la lógica del gesto.

```
// Contenido para añadir/modificar en: src/app/components/task-item/task-item.

import { Component, Input, ViewChild, ElementRef, AfterViewInit } from '@angul
// 1. Importamos las herramientas para gestos
import { GestureController, Gesture } from '@ionic/angular/standalone';
// ... otros imports

@Component({
  // ...
```

```
})
export class TaskItemComponent implements AfterViewInit {

  @Input() tarea!: Tarea;
  // 2. Obtenemos una referencia al elemento #card del HTML
  @ViewChild('card', { read: ElementRef }) card!: ElementRef;

  // 3. Inyectamos el GestureController
  constructor(private gestureCtrl: GestureController) { }

  // 4. Este método se ejecuta cuando la vista ya está lista
  ngAfterViewInit() {
    // 5. Creamos el gesto
    const gesture: Gesture = this.gestureCtrl.create({
      el: this.card.nativeElement, // El elemento al que se aplica el gesto
      gestureName: 'swipe-delete',
      threshold: 15, // Mínimo de píxeles para que el gesto empiece
      onMove: ev => {
        // Mueve la tarjeta horizontalmente siguiendo el dedo del usuario
        // Solo permitimos mover hacia la izquierda (valor negativo)
        if (ev.deltaX < 0) {
          this.card.nativeElement.style.transform = `translateX(${ev.deltaX}p›
        }
      },
      onEnd: ev => {
        // Al levantar el dedo, decidimos qué hacer
        const swipeThreshold = this.card.nativeElement.offsetWidth * 0.4; // 4

        if (Math.abs(ev.deltaX) > swipeThreshold) {
          // Si se ha deslizado lo suficiente, lo movemos fuera de la pantalla
          this.card.nativeElement.style.transform = `translateX(-100%)`;
          console.log(`¡Eliminar tarea ${this.tarea.id}!`);
          // Aquí iría la lógica para eliminar la tarea de verdad
        } else {
          // Si no, devolvemos la tarjeta a su posición original con una anima
          this.card.nativeElement.style.transition = '.2s ease-out';
          this.card.nativeElement.style.transform = `translateX(0px)`;
        }
      },
    });

    // 6. Activamos el gesto
    gesture.enable(true);
  }
```

```
        // ...
    }
```

Si ahora ejecutas la aplicación, podrás deslizar cada una de las tarjetas de tarea hacia la izquierda. Verás cómo sigue a tu dedo y, si la deslizas lo suficiente, se moverá completamente fuera de la pantalla y aparecerá un mensaje en la consola. ¡Acabas de implementar tu primera interacción táctil avanzada!

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.12. Dando Vida a la Interfaz con Animaciones

3.12. Dando Vida a la Interfaz con Animaciones

Una interfaz de usuario estática puede sentirse aburrida y poco reactiva. Las **animaciones**, incluso las más sutiles, pueden mejorar drásticamente la experiencia de usuario, guiando la atención, proporcionando feedback visual y haciendo que la aplicación se sienta más viva y pulida.

Ionic, al estar construido sobre las tecnologías web, nos permite usar tanto las animaciones estándar de CSS como su propia API de animaciones, que es mucho más potente y está optimizada para el rendimiento en dispositivos móviles.

Enfoque 1: Animaciones Sencillas con Transiciones CSS

Para cambios de estado simples, como un cambio de color o tamaño al pasar el ratón por encima, la forma más fácil de añadir una animación es con la propiedad `transition` de CSS.

Imagina que queremos que nuestras tarjetas de tarea (`task-item`) se agranden ligeramente al pasar el cursor por encima. Simplemente tendríamos que añadir unas pocas líneas en su archivo de estilos.

Ejemplo en `task-item.component.scss`:

```
/* Contenido para añadir en: src/app/components/task-item/task-item.component.

ion-card {
  /* Le decimos que cualquier cambio en la propiedad 'transform' dure 0.2 segu
  transition: transform 0.2s ease-in-out;
}

/* Cuando el ratón está encima de la tarjeta (:hover) */
ion-card:hover {
  /* La hacemos un 2% más grande */
```

```
    transform: scale(1.02);
  }
```

Con solo este código, hemos añadido una micro-interacción que hace la interfaz más agradable.

Enfoque 2: La API de Animaciones de Ionic (El Método Profesional)

Para animaciones más complejas (como hacer que un elemento aparezca, se mueva por la pantalla o se encadene con otras animaciones), Ionic nos proporciona la **`AnimationController`**. Esta herramienta nos da un control total sobre las animaciones desde nuestro código TypeScript.

El patrón es muy similar al que ya hemos visto con los Toasts y Alerts:

1. Importar e inyectar el `AnimationController` en el constructor.
2. Obtener una referencia al elemento del HTML que queremos animar (con `@ViewChild`).
3. Usar el `animationCtrl.create()` para definir la animación (qué elemento se anima, qué propiedades cambian, duración, etc.).
4. Llamar al método `.play()` de la animación para ejecutarla.

Ejemplo Práctico: Animando la Aparición de una Tarjeta

Vamos a hacer que la tarjeta de perfil que creamos en la sección 3.1 aparezca con una animación de fundido y deslizamiento cuando la página se carga.

1. La Vista en `home.page.html`

Añadimos una variable de referencia de plantilla (`#profileCard`) a nuestra `<ion-grid>` para poder seleccionarla desde el código.

```
<!-- Archivo: src/app/home/home.page.html -->

<ion-grid #profileCard>
  <!-- ... el contenido de la rejilla con el perfil ... -->
</ion-grid>
```

2. La Lógica en `home.page.ts`

Aquí es donde definimos y ejecutamos la animación.

```
// Contenido para añadir/modificar en: src/app/home/home.page.ts

import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core
// 1. Importamos las herramientas para animaciones
import { AnimationController, Animation } from '@ionic/angular/standalone';
// ... otros imports

@Component({
  // ...
})
export class HomePage implements AfterViewInit {

  // 2. Obtenemos la referencia al elemento #profileCard del HTML
  @ViewChild('profileCard', { read: ElementRef }) profileCard!: ElementRef;
  private animacion!: Animation;

  // 3. Inyectamos el AnimationController
  constructor(private animationCtrl: AnimationController) { }

  // 4. Este método se ejecuta cuando la vista ya está lista
  ngAfterViewInit() {
    // 5. Creamos la animación
    this.animacion = this.animationCtrl
      .create()
      .addElement(this.profileCard.nativeElement) // El elemento a animar
      .duration(800) // Duración de 800ms
      .fromTo('transform', 'translateX(-100px)', 'translateX(0px)') // Mover d
      .fromTo('opacity', '0', '1'); // Cambiar opacidad de 0 a 1 (fundido)

    // 6. Ejecutamos la animación
    this.animacion.play();
  }
}
```

Si ahora recargas la página, verás cómo la tarjeta del perfil aparece con un suave deslizamiento desde la izquierda y un efecto de fundido. La API de animaciones de Ionic es increíblemente potente y te permite encadenar animaciones complejas (`.addAnimation()`), agruparlas (`.groupBy()`) y mucho más.

> ✅ **Práctica Rápida**

> Intenta crear una animación para el botón de "Contactar" de tu perfil. Haz que, al cargarse la página, el botón aparezca desde abajo y con un fundido (pista: usa la propiedad `translateY` en lugar de `translateX`).

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.13. Reto: Maqueta y Estiliza tu Proyecto

3.13. Reto: Maqueta y Estiliza tu Proyecto

¡Es hora de que tu proyecto personal cobre vida! En esta tarea, aplicarás todas las técnicas de diseño de interfaces que hemos visto en la Unidad 3 para transformar el esqueleto de tu aplicación en una interfaz atractiva, responsiva y funcional.

Esta tarea se divide en dos niveles: un reto básico obligatorio para aprobar, y varios retos de mejora que serán necesarios para poder optar a las calificaciones más altas.

> ### ✅ Reto Básico (Requisitos Mínimos para Aprobar)
>
> El objetivo es aplicar los conceptos de maquetación, theming y formularios para construir una de las vistas principales de tu aplicación. Para ello, debes:
>
> 1. **Maquetación con Grid Responsivo:** Aplica el sistema de `<ion-grid>` a la página principal de tu proyecto para crear una estructura de al menos dos columnas. Asegúrate de que la maquetación sea **responsiva**, apilando las columnas en pantallas pequeñas (usando `size="12"`) y mostrándolas una al lado de la otra en pantallas más grandes (usando `size-md` o similar).
> 2. **Incluir Imágenes e Iconos:** En tu maquetación, debes incluir al menos una imagen relevante para tu temática (usando `<ion-img>`, `<ion-avatar>` o `<ion-thumbnail>`) y al menos un icono (usando `<ion-icon>`). No olvides registrarlos en `app.component.ts`.
> 3. **Personalización de Colores:** Define una paleta de colores para tu proyecto. En el archivo `src/theme/variables.scss`, modifica al menos los colores `--ion-color-primary` y `--ion-color-secondary`.
> 4. **Crear un Formulario Básico:** En una parte de tu página, crea un pequeño formulario con un mínimo de **dos campos de entrada diferentes** (ej: un `<ion-input>` y un `<ion-select>`).
> 5. **Enlazar el Formulario con `[(ngModel)]`:** En el archivo `.ts` de tu página, crea las propiedades necesarias y enlázalas a los campos del formulario usando el "two-way data binding". No olvides importar el `FormsModule`.

6. **Añadir un Botón de Envío con Notificación:** Añade un `<ion-button>` para "guardar" los datos del formulario. Al hacer clic en él, debe llamar a un método que muestre un **`<ion-toast>`** con un mensaje de confirmación.

🚀 Retos para la Máxima Calificación (Bien / Excelente)

Completar el reto básico es suficiente para aprobar, pero para obtener una calificación de **Bien** o **Excelente**, se espera que demuestres iniciativa investigando e implementando una o varias de las siguientes mejoras.

- **Mejora 1 (Alineación):** En alguna de las columnas de tu grid, utiliza las **clases de utilidad de Flexbox** (`.ion-justify-content-...`, `.ion-align-items-...`) para crear una alineación más compleja o precisa del contenido.
- **Mejora 2 (Theming Avanzado):** No te conformes con el modo oscuro por defecto. En tu archivo `variables.scss`, personaliza los colores principales (`--ion-color-primary`, etc.) y los de fondo (`--ion-background-color`) dentro del bloque `@media (prefers-color-scheme: dark)`.
- **Mejora 3 (Identidad Visual):** Personaliza la **tipografía** de tu aplicación integrando una fuente de Google Fonts, como se explica en la sección 3.7.
- **Mejora 4 (Interacción):** En lugar de guardar los datos del formulario directamente, haz que el botón "Guardar" primero muestre un **`<ion-alert>`** de confirmación con "Aceptar" y "Cancelar". El `ion-toast` solo debe aparecer si el usuario pulsa "Aceptar").
- **Mejora 5 (UX Avanzada):** Implementa un estado de carga en tu página. Haz que al entrar se muestren durante 2-3 segundos unos **`<ion-skeleton-text>`** que imiten tu maquetación, y que después aparezca el contenido real.
- **Mejora 6 (UX Avanzada):** Usando el **`AnimationController`**, añade una animación de entrada a uno de los elementos principales de tu página (como la rejilla o una tarjeta) para que aparezca con un efecto de fundido o deslizamiento.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**

Desarrollo de Aplicación Híbridas

3.14. Recursos Adicionales

3.14. Recursos Adicionales

¡Felicidades por completar la Unidad 3! Has aprendido los fundamentos para crear interfaces de usuario atractivas y funcionales con Ionic. Sin embargo, solo hemos arañado la superficie de lo que es posible.

La documentación oficial de Ionic es tu mejor aliada para profundizar en cada concepto y descubrir todas las opciones de personalización. A continuación, te dejamos una lista de enlaces directos a las secciones más relevantes que hemos tratado en esta unidad.

Enlaces a la Documentación Oficial de Ionic

- **Maquetación y Grid:**
  - Sistema de Rejilla (Grid) <https://ionicframework.com/docs/layout/grid> : La documentación completa sobre `ion-grid`, `ion-row` y `ion-col`, incluyendo todas las propiedades responsivas.
  - Utilidades CSS <https://ionicframework.com/docs/layout/css-utilities> : Aquí encontrarás todas las clases de utilidad para Flexbox (`.ion-justify-content-...`), espaciado (`.ion-padding`), alineación de texto y mucho más.
- **Estilos y Theming:**
  - Conceptos Básicos de Theming <https://ionicframework.com/docs/theming/basics> : Una introducción a cómo Ionic gestiona los estilos.
  - Colores <https://ionicframework.com/docs/theming/colors> : La guía oficial sobre las variables de color (`--ion-color-primary`, etc.).
  - Modo Oscuro (Dark Mode) <https://ionicframework.com/docs/theming/dark-mode> : Todo lo que necesitas saber para personalizar el tema oscuro.
- **Iconos, Fuentes e Imágenes:**
  - Ionicons <https://ionic.io/ionicons> : Catálogo oficial de iconos.
  - ion-img, ion-avatar, ion-thumbnail <https://ionicframework.com/docs/api/img> : Documentación de los componentes de imagen.
- **Componentes Interactivos:**

    - Componentes de Formulario (Inputs) <https://ionicframework.com/docs/components#inputs> : La sección de la documentación con todos los componentes de formulario (`ion-input`, `ion-select`, `ion-toggle`, etc.).
    - Componentes de Overlay <https://ionicframework.com/docs/components#overlays> : La sección con los componentes de ventanas emergentes (`ion-toast`, `ion-alert`, `ion-modal`...).
    - Skeleton Text <https://ionicframework.com/docs/api/skeleton-text> : Documentación sobre los indicadores de carga.
- **Interacciones Avanzadas:**
    - Gestos Táctiles (Gestures) <https://ionicframework.com/docs/utilities/gestures> : Guía completa sobre cómo usar el `GestureController`.
    - Animaciones (Animations) <https://ionicframework.com/docs/utilities/animations> : Documentación oficial de la API de animaciones de Ionic.

Te recomendamos encarecidamente que guardes estos enlaces y los consultes a menudo, especialmente cuando estés trabajando en el reto de esta unidad y en tu proyecto personal.

Obra publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0 <http://creativecommons.org/licenses/by-sa/4.0/>**